

EXP NO: 6	FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS
------------------	--

AIM:

To implement a **Feedforward Neural Network (FNN)** for classification and a **Convolutional Neural Network (CNN)** for image classification using TensorFlow/Keras.

ALGORITHM:

1. Feedforward Neural Network (FNN)

1. Define the network architecture.
2. Select activation functions and loss function.
3. Choose an optimizer.
4. Train the model using the training data.
5. Evaluate the model using accuracy.

2. Convolutional Neural Network (CNN)

1. Apply convolution and ReLU layers for feature extraction.
2. Apply pooling to reduce feature dimensions.
3. Flatten the feature maps.
4. Add fully connected and output layers.
5. Train and evaluate the CNN using accuracy.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist

tf.keras.utils.disable_interactive_logging()

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

```
x_train = x_train.reshape(-1, 784) / 255.0
```

```
x_test = x_test.reshape(-1, 784) / 255.0
```

```
fnn = keras.Sequential([  
    layers.Input(shape=(784,)),  
    layers.Dense(128, activation='relu'),  
    layers.Dropout(0.2),  
    layers.Dense(64, activation='relu'),  
    layers.Dense(10, activation='softmax')  
])
```

```
fnn.compile(optimizer='adam',  
            loss='sparse_categorical_crossentropy',  
            metrics=['accuracy'])
```

```
fnn.fit(x_train, y_train, epochs=5, validation_split=0.1)
```

```
loss, acc = fnn.evaluate(x_test, y_test, verbose=0)  
print("FNN Accuracy:", acc)
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train.reshape(-1, 28, 28, 1) / 255.0
```

```
x_test = x_test.reshape(-1, 28, 28, 1) / 255.0
```

```
cnn = keras.Sequential([  
    layers.Input(shape=(28, 28, 1)),  
    layers.Conv2D(32, (3,3), activation='relu'),  
    layers.MaxPooling2D((2,2)),  
    layers.Conv2D(64, (3,3), activation='relu'),  
    layers.MaxPooling2D((2,2)),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(10, activation='softmax')  
])
```

```
cnn.compile(optimizer='adam',  
            loss='sparse_categorical_crossentropy',  
            metrics=['accuracy'])
```

```

cnn.fit(x_train, y_train, epochs=5, validation_split=0.1)

loss, acc = cnn.evaluate(x_test, y_test, verbose=0)
print("CNN Accuracy:", acc)

pred = np.argmax(cnn.predict(x_test[:10], verbose=0), axis=1)

plt.figure(figsize=(10,4))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
    plt.title(f"True: {y_test[i]} \n Pred: {pred[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()

```

OUTPUT:

```

--- Part 1: Building a Simple Feedforward Neural Network ---

Original FNN training data shape: (60000, 28, 28)
Original FNN test data shape: (10000, 28, 28)
Flattened & Normalized FNN training data shape: (60000, 784)
Flattened & Normalized FNN test data shape: (10000, 784)

--- FNN Model Summary ---

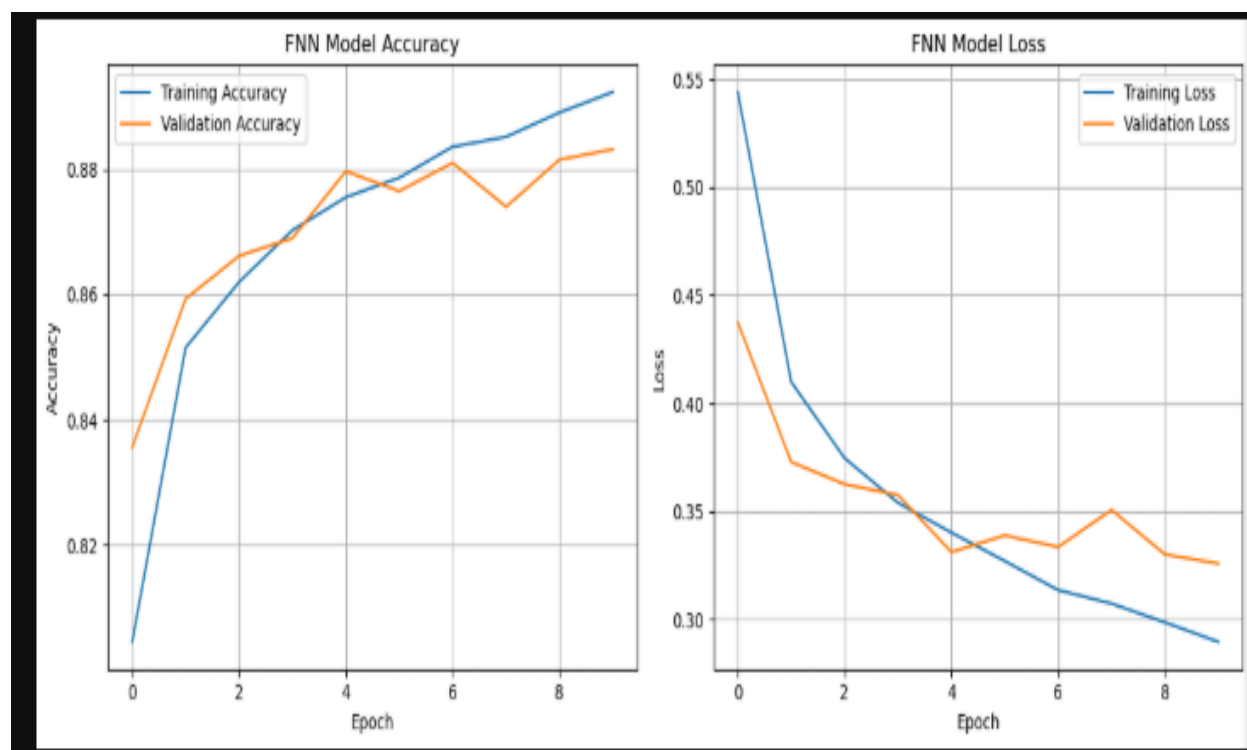
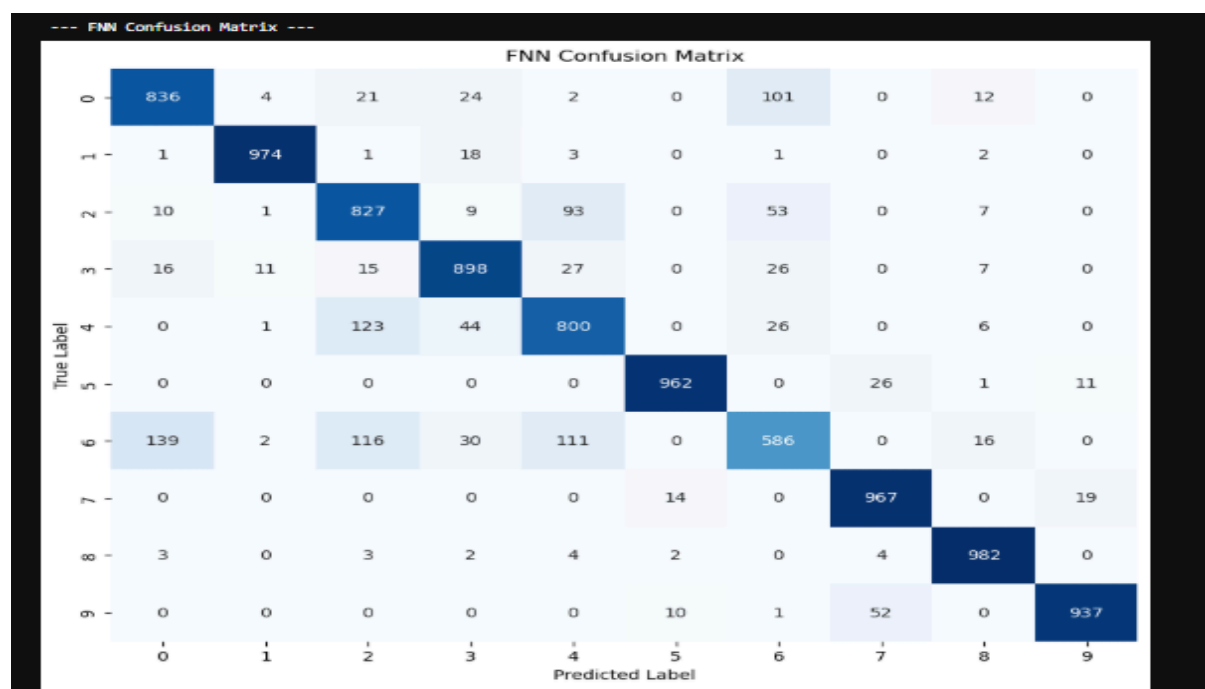
--- Training FNN Model ---

--- Evaluating FNN Model ---
FNN Test Loss: 0.3462
FNN Test Accuracy: 0.8769

--- FNN Classification Report ---

```

	precision	recall	f1-score	support
0	0.83	0.84	0.83	1000
1	0.98	0.97	0.98	1000
2	0.75	0.83	0.79	1000
3	0.88	0.90	0.89	1000
4	0.77	0.80	0.78	1000
5	0.97	0.96	0.97	1000
6	0.74	0.59	0.65	1000
7	0.92	0.97	0.94	1000
8	0.95	0.98	0.97	1000
9	0.97	0.94	0.95	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000



```

--- Part 2: Implementing a CNN ---

Original CNN training data shape: (60000, 28, 28)
Original CNN test data shape: (10000, 28, 28)
Reshaped & Normalized CNN training data shape: (60000, 28, 28, 1)
Reshaped & Normalized CNN test data shape: (10000, 28, 28, 1)

--- CNN Model Summary ---

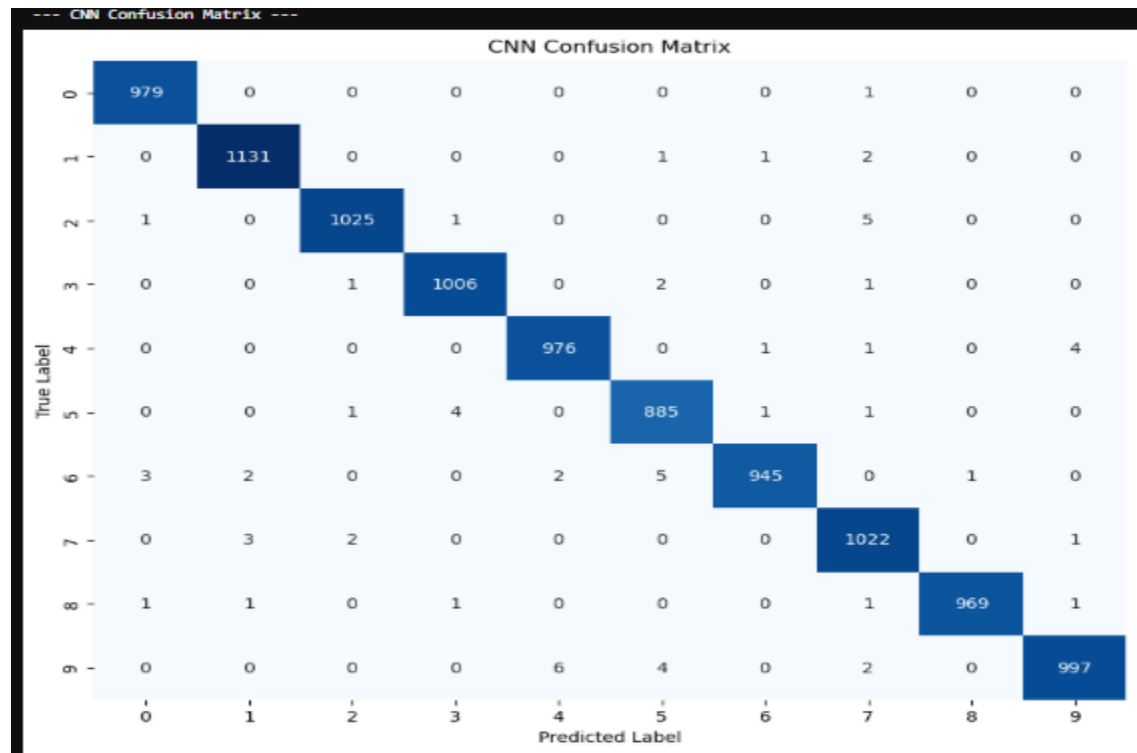
--- Training CNN Model ---

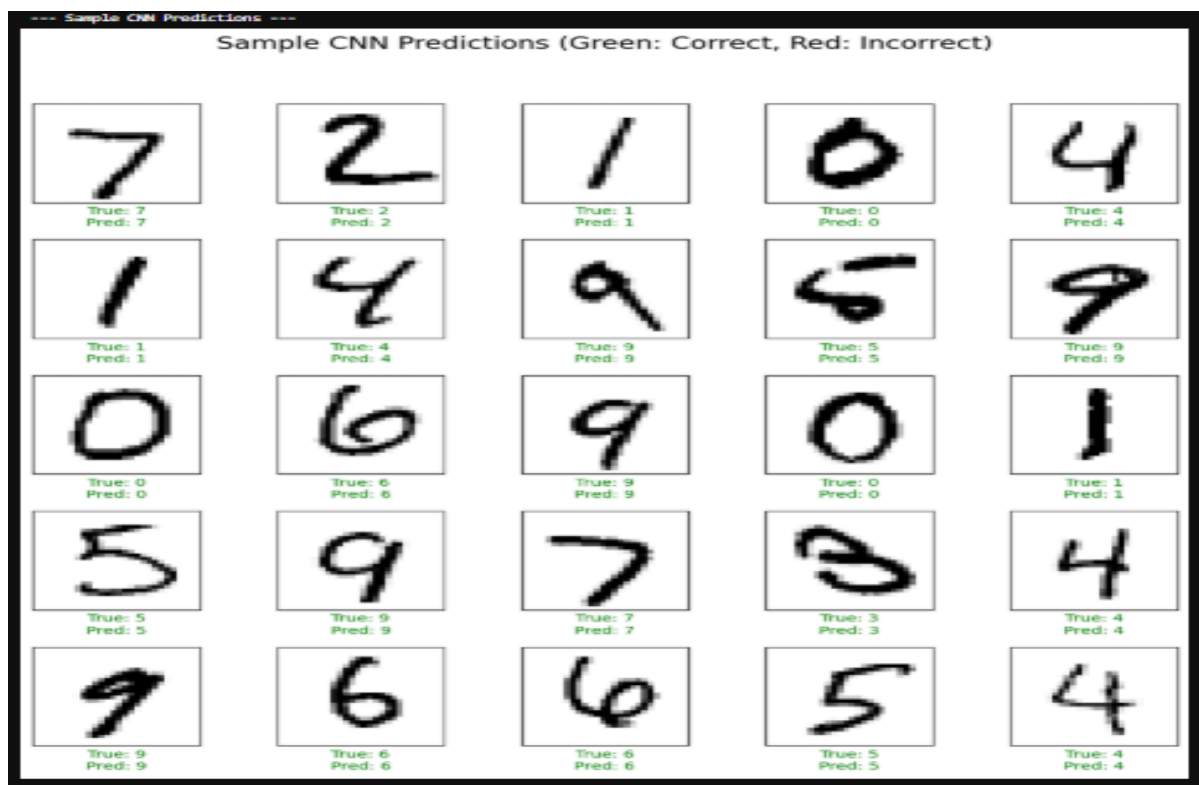
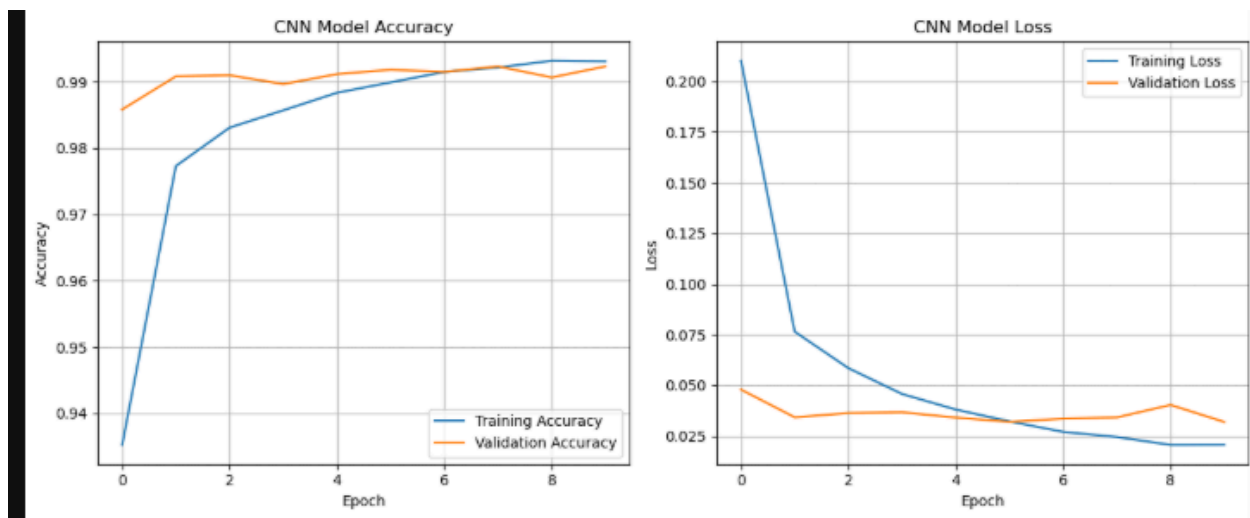
--- Evaluating CNN Model ---
CNN Test Loss: 0.0249
CNN Test Accuracy: 0.9935

--- CNN Classification Report ---

```

	precision	recall	f1-score	support
0	0.99	1.00	1.00	980
1	0.99	1.00	1.00	1135
2	1.00	0.99	0.99	1032
3	0.99	1.00	1.00	1010
4	0.99	0.99	0.99	982
5	0.99	0.99	0.99	892
6	1.00	0.99	0.99	958
7	0.99	0.99	0.99	1028
8	1.00	0.99	1.00	974
9	0.99	0.99	0.99	1009
accuracy			0.99	10000
macro avg	0.99	0.99	0.99	10000
weighted avg	0.99	0.99	0.99	10000





RESULT:

The FNN and CNN models were successfully implemented and evaluated. FNN performed well for classification, while CNN effectively extracted image features and achieved better performance for image classification.